

Long Answer type questions

1. Define Web Applications. Explain tools for website creation.

Definition:

A **web application** is software that runs on a web server and is accessed through a browser using the Internet or an intranet. Unlike traditional desktop applications, web apps are platform-independent, require no installation, and are easily accessible from anywhere.

Explanation:

Web applications integrate:

- **Client-side technologies:** HTML, CSS, and JavaScript for structure, style, and interactivity.
- **Server-side technologies:** PHP, Python (Django/Flask), Node.js, or ASP.NET handle logic and connect to databases.
- **Databases:** MySQL, PostgreSQL, and MongoDB store and manage application data.

Web apps are commonly used for **email (Gmail), online banking, e-commerce, and social media**.

Tools Used for Website Creation:

1. **Text Editors/IDEs:** Visual Studio Code, Sublime Text, Atom.
2. **Design Tools:** Figma, Adobe XD, Canva.
3. **Frameworks:**
 - Frontend: React, Angular, Vue.js
 - Backend: Laravel, Django, Express.js
4. **Web Servers:** Apache, Nginx, IIS.
5. **Databases:** MySQL, PostgreSQL, Oracle, MongoDB.
6. **Version Control:** Git and GitHub for code management.
7. **Testing Tools:** Selenium, Postman.
8. **Deployment Tools:** Netlify, AWS, Heroku.

Code Example (Grey Box):

```
# Project Folder Structure
/project-root
  /public          # static files
  /src             # source code
  /views           # templates
  index.php
  style.css
```

package.json

Conclusion:

Web applications are the foundation of the modern Internet. With a combination of tools, frameworks, and languages, developers create scalable, responsive, and interactive websites.

2. Purpose and ways of creating lists in HTML

Definition:

HTML lists organize content into ordered or unordered groups, making information more structured and easier to read.

Types of Lists:

1. **Ordered List ():** Displays items with numbers (1, 2, 3).
2. **Unordered List ():** Displays items with bullets (●, ○, ■).
3. **Definition List (<dl>):** Displays terms and definitions.

Code Example (Grey Box):

```
<h3>Ordered List</h3>
<ol type="1">
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ol>

<h3>Unordered List</h3>
<ul style="list-style-type: square;">
  <li>Frontend</li>
  <li>Backend</li>
  <li>Database</li>
</ul>

<h3>Definition List</h3>
<dl>
  <dt>HTTP</dt>
  <dd>HyperText Transfer Protocol</dd>
  <dt>URL</dt>
  <dd>Uniform Resource Locator</dd>
</dl>
```

Conclusion:

HTML lists enhance readability, organization, and navigation on web pages. They are also widely used in menus and documentation.

3. Steps for displaying XML documents using CSS

Definition:

CSS can be applied to XML to improve its presentation. Although XML itself only stores data, CSS allows browsers to render that data in a readable format.

Steps:

1. Create an **XML document** that contains structured data.
2. Create a **CSS file** that defines how each XML element should look.
3. Link the CSS file to XML using a **stylesheet declaration**.

Code Example (Grey Box):

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/css" href="note.css"?>
<note>
  <to>John</to>
  <from>Jane</from>
  <body>Hello, this is a styled XML message.</body>
</note>
```

CSS (note.css):

```
note { display: block; border: 1px solid #333; padding: 10px; }
to, from, body { display: block; margin: 5px; }
to { color: darkblue; font-weight: bold; }
```

Conclusion:

By linking CSS, XML data becomes visually appealing and more understandable without altering its structure.

4. Short Notes on: WWW, ISP, and Web Servers

WWW (World Wide Web):

A global system of interconnected documents and resources accessible via the Internet using web browsers. It operates on **HTTP** and uses **URLs** for locating web content.

ISP (Internet Service Provider):

An organization that provides users with Internet connectivity and related services such as email, hosting, and broadband.

Examples: Airtel, Jio, BSNL.

Web Servers:

A web server stores, processes, and delivers web content through HTTP.

Examples include **Apache**, **Nginx**, and **Microsoft IIS**.

Conclusion:

WWW, ISP, and Web Servers are the backbone of Internet communication, connecting users, data, and devices globally.

5. Explain the Table Tag in detail with an example using rowspan and colspan. Also use image and anchor tag in example.

Definition:

The `<table>` tag is used in HTML to display information in rows and columns. Tables help organize structured data such as schedules or reports.

Explanation:

- `<tr>` defines a row.
- `<th>` defines a header cell.
- `<td>` defines a data cell.
- `rowspan` merges cells vertically; `colspan` merges them horizontally.

Code Example (Grey Box):

```
<table border="1" cellpadding="6">
  <caption>Resource Table</caption>
  <tr>
    <th rowspan="2">Logo</th>
    <th colspan="2">Details</th>
  </tr>
```

```

<tr>
  <th>Name</th>
  <th>Link</th>
</tr>
<tr>
  <td></td>
  <td>HTML Tutorial</td>
  <td><a href="https://developer.mozilla.org/" target="_blank">Visit
MDN</a></td>
</tr>
</table>

```

Explanation of Example:

- The `rowspan` attribute merges the “Logo” cell over two rows.
- The `colspan` attribute merges the “Details” header over two columns.
- The image (`<img>`) and anchor (`<a>`) tags add multimedia and interactivity.

Conclusion:

Tables are essential for representing structured data in a clear, visual format. Combining `rowspan`, `colspan`, and multimedia elements makes data presentation more attractive and functional.

6. What is CSS? Explain different types of CSS with advantages and disadvantages.

Definition:

CSS (Cascading Style Sheets) defines the presentation of web pages, including layout, colors, and fonts. It separates structure (HTML) from style, enabling flexible and consistent design.

Types of CSS:

1. **Inline CSS** — written within an HTML element using the `style` attribute.
 -  *Advantage:* Quick and specific.
 -  *Disadvantage:* Hard to maintain; duplicates style rules.
2. **Internal CSS** — written within `<style>` tags inside the HTML `<head>`.
 -  *Advantage:* Good for single-page styling.
 -  *Disadvantage:* Not reusable across pages.
3. **External CSS** — linked through a `.css` file using `<link>`.
 -  *Advantage:* Reusable, efficient, and easy to maintain.
 -  *Disadvantage:* Needs an extra HTTP request.

Code Example (Grey Box):

```
<!-- Inline -->
<p style="color:red;">This is inline styled text.</p>

<!-- Internal -->
<head>
  <style>
    h1 { color: green; text-align: center; }
  </style>
</head>

<!-- External -->
<link rel="stylesheet" href="style.css">
```

Conclusion:

External CSS is the best practice for large websites, as it promotes consistency, separation of concerns, and better load performance.

7. Compare HTML and XML. Explain internal and external DTD with example.

Comparison:

Feature	HTML	XML
Purpose	Structure + presentation	Data storage and transport
Tags	Predefined	User-defined
Case Sensitivity	Not case-sensitive	Case-sensitive
Errors	Browser-tolerant	Strict (well-formed)
End Tags	Optional for some	Mandatory for all

Definition (DTD):

DTD (Document Type Definition) defines rules and structure for an XML document. It validates that elements follow proper hierarchy and syntax.

Internal DTD Example:

```
<?xml version="1.0"?>
<!DOCTYPE note [
  <!ELEMENT note (to,from,body)>
  <!ELEMENT to (#PCDATA)>
  <!ELEMENT from (#PCDATA)>
```

```
<!ELEMENT body (#PCDATA)>
]>
<note>
  <to>John</to>
  <from>Jane</from>
  <body>Hello XML!</body>
</note>
```

External DTD Example:

```
<?xml version="1.0"?>
<!DOCTYPE note SYSTEM "note.dtd">
<note>
  <to>John</to>
  <from>Jane</from>
  <body>Hello from external DTD!</body>
</note>
```

Advantages of DTD:

- Ensures consistency and structure.
- Validates XML syntax.
- Promotes reuse in large projects.

Conclusion:

HTML focuses on how data looks, while XML defines what data means. DTD enforces rules that make XML structured and reliable.

8. Explain the purpose and various ways of creating lists in HTML documents (additional).

Theory:

Lists are used to present grouped or hierarchical data in a readable and organized manner. They enhance structure and improve navigation.

Types of Lists:

1. **Ordered List ():** For ranked items (steps, priorities).
2. **Unordered List ():** For unordered information (features).
3. **Nested Lists:** Lists inside lists.
4. **Definition List (<dl>):** For term–definition pairs.

Code Example (Grey Box):

```
<ul>
  <li>Frontend
    <ol>
      <li>HTML</li>
      <li>CSS</li>
      <li>JavaScript</li>
    </ol>
  </li>
  <li>Backend
    <ul>
      <li>PHP</li>
      <li>MySQL</li>
    </ul>
  </li>
</ul>
```

Conclusion:

HTML lists provide semantic structure and flexibility. Nested lists are ideal for hierarchical menus or outlines.

9. Explain various tools for website creation in detail.

Theory:

Website creation involves multiple stages — planning, designing, developing, testing, and deploying — each supported by specialized tools.

Tools by Category:

1. **Planning:** MindMeister, Trello, Notion.
2. **Design:** Adobe XD, Figma, Canva for prototyping and mockups.
3. **Development:**
 - Frontend: VS Code, React, Angular.
 - Backend: Node.js, Laravel, Django.
4. **Testing:** Selenium, Postman, Lighthouse.
5. **Deployment:** GitHub Pages, Netlify, AWS, Heroku.
6. **Version Control:** Git + GitHub for collaboration.
7. **Analytics & Monitoring:** Google Analytics, Sentry.

Example (Grey Box):

```
Web Dev Workflow
1. Design UI in Figma
2. Develop frontend in React
3. Use Node.js for backend API
4. Test with Postman
```

5. Deploy via Netlify

Conclusion:

Using the right combination of tools ensures quality, scalability, and faster development of modern websites.

10. Explain various steps for displaying XML documents using CSS with suitable examples (additional).

Theory:

Displaying XML with CSS enhances human readability. Unlike HTML, XML doesn't have built-in style definitions, so CSS must be manually linked.

Steps:

1. Create an XML file containing structured data.
2. Create a CSS file describing the appearance of tags.
3. Link them using the `xml-stylesheet` instruction.

Code Example (Grey Box):

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/css" href="books.css"?>
<library>
  <book>
    <title>Web Technologies</title>
    <author>John Doe</author>
  </book>
</library>
```

CSS (books.css):

```
library { display: block; border: 1px solid gray; padding: 8px; }
book { display: block; margin: 10px; }
title { font-weight: bold; color: darkblue; }
author { color: green; }
```

Conclusion:

XML with CSS allows meaningful data to be presented neatly for visual interpretation, bridging the gap between raw data and formatted content.

11. Define Internet and Explain Various Internet Connections

Definition

The **Internet** is a global network of computers linked together using the **TCP/IP** protocol suite. It allows worldwide communication, data sharing, and resource access.

Types of Internet Connections

1. **Dial-up Connection:**
Uses a telephone line and modem; very slow (56 kbps).
✗ Outdated due to limited bandwidth.
2. **DSL (Digital Subscriber Line):**
Uses phone lines but allows simultaneous phone + Internet.
3. **Cable Connection:**
Delivers Internet via coaxial TV cables; higher speed (up to 1 Gbps).
4. **Fiber Optic (FTTH):**
Transmits light signals; offers extremely high speed (> 1 Gbps).
5. **Wireless (Wi-Fi):**
Uses radio waves; suitable for LAN connections at home/offices.
6. **Mobile (3G/4G/5G):**
Provides mobility; speed and latency depend on generation.
7. **Satellite Internet:**
Ideal for remote areas but has higher latency.

Conclusion

Different Internet connections trade off between **speed, reliability, and cost**. Fiber and 5G offer the best modern performance.

12. Write HTML Code to Create a Seminar Schedule Table

Theory

HTML tables display structured information such as schedules or timetables using rows and columns.

Use `<table>`, `<tr>`, `<th>`, `<td>` for basic structure and attributes like `rowspan/colspan` for cell merging.

Code Example (Grey Box)

```

<table border="1" cellpadding="5">
  <caption>Table 10: Seminar Schedule</caption>
  <tr>
    <th rowspan="2">Time</th>
    <th colspan="2">Session</th>
    <th rowspan="2">Speaker</th>
  </tr>
  <tr>
    <th>Title</th>
    <th>Room</th>
  </tr>
  <tr>
    <td>10:00 AM - 11:00 AM</td>
    <td>Intro to Web Tech</td>
    <td>Lab-1</td>
    <td>Dr. Smith</td>
  </tr>
  <tr>
    <td>11:15 AM - 12:15 PM</td>
    <td>PHP Workshop</td>
    <td>Lab-2</td>
    <td>Mr. John</td>
  </tr>
</table>

```

Conclusion

Tables with captions, headers, and row/column spans make data easier to read and understand.

13. Write PHP Code for Cookies and Session Management

Theory

- **Cookies** store small pieces of data (client-side).
 - **Sessions** maintain user state (server-side).
- Both are essential for login systems, shopping carts, and personalization.

Code Example (Grey Box)

```

<?php
// Set Cookie
setcookie("username", "John", time()+3600, "/");

// Start Session
session_start();
$_SESSION["user_id"] = 101;

// Retrieve Values
echo "Cookie: " . $_COOKIE["username"];
echo "Session ID: " . $_SESSION["user_id"];

```

?>

Conclusion

Cookies persist small data on the client; sessions securely store critical state on the server.

14. Process for Connecting to MySQL Database in PHP

Steps

1. **Establish Connection** using `mysqli_connect()` or `PDO`.
2. **Check Connection Errors.**
3. **Execute Queries.**
4. **Close Connection.**

Code Example (Grey Box)

```
<?php
$conn = mysqli_connect("localhost", "root", "", "college");
if (!$conn) {
    die("Connection failed: ".mysqli_connect_error());
}
echo "Connected successfully";
mysqli_close($conn);
?>
```

Conclusion

A successful MySQL connection is essential for storing and retrieving dynamic web data using PHP.

15. Discuss the Role of PHP in Web Development. Explain Looping Statements with Examples

Role of PHP in Web Development

PHP is a server-side scripting language that powers dynamic websites and applications. It handles form processing, sessions, database operations, and page generation.

Looping Statements in PHP

1. **for loop:** Repeats a block with a counter.

2. **while loop:** Runs while a condition is true.
3. **do-while loop:** Executes at least once.
4. **foreach loop:** Iterates through arrays.

Code Example (Grey Box)

```
<?php
// for loop
for($i=1; $i<=5; $i++){
    echo "Number: $i<br>";
}

// foreach loop
$colors = ["red", "green", "blue"];
foreach($colors as $c){
    echo "Color: $c<br>";
}
?>
```

Conclusion

PHP enables server-side logic and database interaction.
Looping constructs reduce repetition and enhance program efficiency.

16. How to Retrieve Data from MySQL Database Using PHP

Theory

PHP interacts with MySQL to fetch and display database records dynamically on web pages.
This is essential for web apps like login systems, dashboards, and reports.

Steps:

1. Establish a database connection using `mysqli_connect()`.
2. Execute an SQL `SELECT` query.
3. Fetch results using `mysqli_fetch_assoc()`.
4. Display and close the connection.

Code Example (Grey Box):

```
<?php
$conn = mysqli_connect("localhost", "root", "", "college");

if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}
```

```

$result = mysqli_query($conn, "SELECT name, email FROM students");

while($row = mysqli_fetch_assoc($result)) {
    echo "Name: " . $row['name'] . " | Email: " . $row['email'] . "<br>";
}

mysqli_close($conn);
?>

```

Conclusion

Fetching and displaying MySQL data using PHP is simple yet powerful — it forms the foundation of dynamic web content.

17. Explain the Role of GET and POST Methods in PHP

Theory

In PHP, **GET** and **POST** are methods used to send data from a form to a server. They define how form data is transmitted during HTTP requests.

Difference Between GET and POST:

Aspect	GET	POST
Visibility	Data visible in URL	Data hidden in request body
Use Case	Search queries, bookmarks	Login, form submissions
Data Limit	~2048 characters	Virtually unlimited
Security	Less secure	More secure

Code Example (Grey Box):

```

<!-- GET Example -->
<form method="get" action="get_example.php">
    <input type="text" name="search">
    <input type="submit" value="Search">
</form>

<!-- POST Example -->
<form method="post" action="post_example.php">
    <input type="text" name="username">
    <input type="password" name="password">
    <input type="submit" value="Login">
</form>

```

Conclusion

Use **GET** for non-sensitive, retrievable data and **POST** for secure, user-input data submission.

18. What is an Exception? How Can You Handle Multiple Exceptions in PHP?

Definition

An **Exception** in PHP is an object that represents an error or unexpected behavior during program execution.

Handling Multiple Exceptions:

PHP uses `try`, `catch`, and `finally` blocks to manage exceptions.

Code Example (Grey Box):

```
<?php
try {
    $num = 0;
    if ($num == 0) {
        throw new Exception("Division by zero not allowed");
    }
    $result = 10 / $num;
}
catch (DivisionByZeroError $e) {
    echo "Math Error: ".$e->getMessage();
}
catch (Exception $e) {
    echo "General Error: ".$e->getMessage();
}
finally {
    echo "<br>Process completed.";
}
?>
```

Conclusion

Exception handling ensures program stability by managing runtime errors gracefully without crashing the application.

19. Create a Registration Form in PHP Using Text Boxes, Buttons, Radio Buttons, and Checkboxes

Theory

Registration forms collect user data like name, gender, hobbies, and email. HTML handles structure, and PHP processes and validates the input.

Code Example (Grey Box):

```
<form method="post" action="register.php">
  Name: <input type="text" name="name"><br><br>
  Gender:
  <input type="radio" name="gender" value="Male">Male
  <input type="radio" name="gender" value="Female">Female<br><br>
  Hobbies:
  <input type="checkbox" name="hobbies[]" value="Reading">Reading
  <input type="checkbox" name="hobbies[]" value="Music">Music<br><br>
  <input type="submit" value="Register">
</form>
```

register.php

```
<?php
if($_SERVER["REQUEST_METHOD"] == "POST") {
    $name = $_POST['name'];
    $gender = $_POST['gender'];
    $hobbies = implode(", ", $_POST['hobbies']);
    echo "Name: $name<br>Gender: $gender<br>Hobbies: $hobbies";
}
?>
```

Conclusion

A registration form demonstrates PHP's ability to process and display user inputs dynamically.

20. Write a Program to Arrange the Elements of an Array Using PHP

Theory

Arrays can be sorted alphabetically or numerically using built-in PHP functions.

Sorting Functions:

- `sort()` → ascending order
- `rsort()` → descending order
- `asort()` → associative array by value
- `ksort()` → associative array by key

Code Example (Grey Box):

```
<?php
$numbers = [5, 3, 9, 1, 7];
sort($numbers);
echo "Sorted Array: ";
print_r($numbers);
?>
```

Conclusion

Sorting arrays simplifies data handling and display, especially in tables or lists.

21. Write a Program to Generate the First 10 Fibonacci Numbers Using PHP

Theory

The **Fibonacci series** is a sequence where each number is the sum of the previous two:
0, 1, 1, 2, 3, 5, 8, 13, 21, 34...

PHP can generate this series easily using a loop.

Code Example (Grey Box):

```
<?php
$a = 0;
$b = 1;
echo "Fibonacci Series: ";
for ($i = 0; $i < 10; $i++) {
    echo $a . " ";
    $next = $a + $b;
    $a = $b;
    $b = $next;
}
?>
```

Explanation

- `$a` and `$b` represent the previous two numbers.
- Each iteration prints the current number and updates `$a` and `$b`.

Conclusion

This simple iterative approach avoids recursion and efficiently prints the first 10 Fibonacci numbers.

22. Write a Program to Find the Factorial of a Given Number Using Functions in PHP

Theory

The **factorial (n!)** of a number n is the product of all integers from 1 to n . Mathematically, $n! = n \times (n-1) \times (n-2) \times \dots \times 1$

PHP allows both **iterative** and **recursive** methods to compute factorial.

Code Example (Grey Box):

```
<?php
function factorial($n) {
    if ($n <= 1)
        return 1;
    else
        return $n * factorial($n - 1);
}

$num = 5;
echo "Factorial of $num is: " . factorial($num);
?>
```

Explanation

- The function calls itself recursively with decremented values.
- Base condition stops recursion when $n \leq 1$.

Output

```
Factorial of 5 is: 120
```

Conclusion

Recursive functions make factorial computation elegant and simple in PHP.

23. What Do You Mean by XML Parser? Explain DOM Parser with Its Advantages and Disadvantages

Definition

An **XML Parser** reads and processes XML documents, converting them into data structures that programs can use.

Types of XML Parsers:

1. **DOM (Document Object Model) Parser:**
Loads the entire XML into memory and represents it as a tree.
2. **SAX (Simple API for XML) Parser:**
Reads XML sequentially without loading the entire file.

DOM Parser Example (Grey Box):

```
<?php
$doc = new DOMDocument();
$doc->load("books.xml");

$books = $doc->getElementsByTagName("book");
foreach($books as $book) {
    $title = $book->getElementsByTagName("title")->item(0)->nodeValue;
    echo "Title: " . $title . "<br>";
}
?>
```

Advantages of DOM Parser:

- ✓ Easy to navigate and modify XML structure.
- ✓ Ideal for small to medium XML files.

Disadvantages of DOM Parser:

- ✗ Consumes more memory for large XML documents.
- ✗ Slower performance due to full file loading.

Conclusion

The DOM Parser provides a structured, easy-to-manage interface for XML manipulation, though it's memory-intensive for large datasets.

Short Question Answers

a) Define IP address.

An IP address (Internet Protocol address) is a unique logical identifier assigned to a device on a TCP/IP network so it can be located and communicated with. IPv4 uses 32 bits and is written as four decimal octets (e.g., 192.168.1.10), while IPv6 uses 128 bits and hexadecimal groups (e.g., 2001:db8::1), providing a vastly larger address space and built-in features like autoconfiguration and scoped addressing.

b) Comparison of HTML vs. DHTML.

HTML builds static documents that render the same until reloaded; DHTML (Dynamic HTML) is not a separate language but a combination of HTML + CSS + JavaScript + DOM scripting to change styles, positions, and content at runtime without page refreshes. DHTML enables effects such as show/hide panels, animations, form validation, and dynamic menus, typically driven by event handlers and style changes.

c) Checkbox and Radio Element

Checkboxes (`<input type='checkbox'>`) represent independent boolean options allowing multiple selections; they can share the same name as an array (e.g., `hobby[]`). Radio buttons (`<input type='radio' name='group'>`) belong to a named group and enforce a single selection within that group. Both support attributes like `checked`, `disabled`, and `value`, and should be paired with `<label>` for accessibility.

d) Frames and Frameset

Frames split the browser viewport into independent panes that load separate documents; historically defined by `<frameset>` with `cols/rows` and child `<frame>` tags. While useful for persistent navigation bars, frames complicate bookmarking, history, and accessibility, and are now obsolete in HTML5 (replaced by CSS layouts, `iframes` for embedded documents, and SPA approaches).

e) HTTP and DNS

HTTP/HTTPS is an application-layer protocol that defines how web clients and servers request and transfer resources (methods like GET/POST, status codes, headers). DNS is a distributed naming system that maps human-friendly domain names to IP addresses via queries to recursive and authoritative name servers. DNS typically precedes HTTP by resolving the host to connect to.

f) DTD in XML

A DTD (Document Type Definition) specifies the legal building blocks of an XML document—its elements, attributes, entities, and their ordering/occurrence. It supports validation: a validating parser checks whether the XML instance conforms to the declared structure, enabling interoperability and data integrity across systems that exchange XML.

g) XML Parser

An XML parser reads XML text and exposes its structure/data to programs. DOM parsers load the entire document tree in memory for random access and modification; SAX parsers stream through the document and trigger events for elements/attributes/text, using minimal memory and suiting large documents and transformation pipelines.

h) Web browsers

A web browser is a client application (e.g., Chrome, Firefox, Edge, Safari) that resolves URLs, initiates DNS and HTTP(S) requests, parses HTML/CSS/JS, builds the DOM/CSSOM, performs layout and painting, executes JavaScript on a single-threaded event loop, enforces security models like same-origin policy, and presents interactive web pages to users.

i) Get and Post methods

GET encodes data in the URL query string, is cacheable, bookmarkable, and intended for safe/idempotent retrieval; it has length limits and should not carry sensitive data. POST places data in the request body, is not bookmarked or cached by default, and is used for form submissions that change server state (create/update). Both send headers and can carry payloads, but semantics differ.

j) Strings in PHP

PHP strings are sequences of bytes; single-quoted strings are literal with minimal escaping, while double-quoted strings support variable interpolation and escape sequences. Common operations include concatenation with '.', measuring (strlen), searching/replacing (strpos/str_replace), slicing (substr), and encoding tasks (htmlspecialchars, json_encode).

k) Write Difference between ol & ul tag

 creates ordered, numbered lists (customizable via type/start/reversed), suitable for sequences and steps. creates unordered, bulleted lists for non-ranked items. Both use for items and can be nested; CSS controls markers via list-style properties.

l) Difference between HTTP and DNS.

HTTP transports hypermedia between clients and servers using methods, status codes, and headers at the application layer; DNS resolves domain names to IPs using a hierarchical, distributed database. DNS operates prior to HTTP to determine the numerical endpoint for the HTTP connection.

m) Explain Absolute URL and Relative URL.

An absolute URL contains the full scheme, host, optional port, and path (e.g., `https://example.org/docs/page.html`), resolving the same from any context. A relative URL omits some components and resolves against the current document's base URL (e.g., `../images/logo.png`), improving portability within a site structure.

n) What is Web Applications?

A web application is an interactive program delivered over HTTP/HTTPS and rendered by a browser, typically using HTML/CSS/JavaScript on the client and a server-side stack to authenticate users, validate input, persist data in databases, and return dynamic content or APIs.

o) Define frameset and frame tag with example.

A frameset document replaces the `<body>` with `<frameset>`, dividing the viewport using rows/cols; each `<frame>` loads a different page. Example:

```
<frameset cols='30%,70%'><frame src='nav.html'><frame  
src='main.html'></frameset>
```

p) Frames and Frameset

Frames are the individual panes; a frameset defines their arrangement. Despite historical use for navigation and content regions, frames are deprecated due to usability, SEO, and accessibility issues; modern sites rely on CSS layouts and JavaScript routing.

q) HTTP and DNS

HTTP is a request-response protocol for transferring resources; DNS is the naming system that resolves domain names to IP addresses, enabling clients to know where to send HTTP requests.

r) DTD in XML

A DTD declares the permissible structure and attributes of XML elements. It may be internal (embedded in the XML with `<!DOCTYPE ... [ ... ]>`) or external (SYSTEM/PUBLIC pointing to a .dtd file), enabling validation across documents.

s) XML Parser

Parsers process XML into accessible structures/events. DOM gives a modifiable tree for random access; SAX/StAX stream events for large or real-time processing; validating parsers additionally check against DTD/XSD.

t) Web browsers

Browsers fetch resources, parse HTML into a DOM, apply CSS to form a render tree, execute JS to modify it, and paint the layout to the screen; they also sandbox content, manage cookies/storage, and implement developer tools for inspection and debugging.

u) Exception handling

Exception handling in PHP uses `try { } catch (Type $e) { } finally { }` to isolate error-prone code paths, separate normal from exceptional control flow, and recover or report meaningfully. Multiple catch blocks allow handling different exception classes; finally ensures cleanup.

v) Write a program to find a factorial of a given number using function in PHP.

A simple recursive function multiplies n by $\text{fact}(n-1)$ until n reaches 1 (or 0). Iterative versions avoid deep recursion for large n .

```
<?php
function fact($n){ return ($n<=1)?1:$n*fact($n-1); }
echo fact(5); // 120
?>
```

w) Sessions and Cookies

Cookies are small name–value pairs stored on the client and automatically sent with requests to the same domain; they can be persistent or session-scoped. Sessions store data on the server and identify the client via a session ID (often in a cookie); sessions are better for sensitive data since it isn't exposed client-side.

x) Associative array in PHP

Associative arrays map keys to values: `$student = ['name'=>'Asha','age'=>21];`. They support operations like `isset`, `foreach ($arr as $k=>$v)`, `array_keys/values`, and functions such as `ksort/arsort` for key/value-based sorting.

y) String functions in PHP

Important functions include `strlen`, `trim`, `strtolower/strtoupper`, `substr`, `strpos`, `str_replace`, `explode/implode` for tokenization, and `sprintf` for formatted output. For multibyte encodings, use `mb_*` variants like `mb_strlen` and `mb_substr`.

z) Write syntax for `setcookies()` function in PHP.

The function sets an HTTP response header to create/modify a cookie on the client. Basic form: `setcookie(name, value, expires, path, domain, secure, httponly)`. Example: `setcookie('user','John', time()+3600, '/')`; Modern PHP also offers `setcookie('user','John', ['expires'=>time()+3600,'path'=>'/','samesite'=>'Lax'])`;

aa) Differentiate between PHP and MySQL

PHP is a general-purpose server-side scripting language used to build application logic, render pages, and handle requests; MySQL is a relational database management system used to store and query structured data using SQL. PHP connects to MySQL (e.g., via `mysqli/PDO`) to perform CRUD operations for web apps.

bb) What do you mean by empty element in XML

An empty element has no content and is represented as a self-closing tag, such as `<br/>`, `<img/>`, or `<line/>`. It can also be written with a separate closing tag (`<tag></tag>`) depending on the parser, but self-closing is conventional in XML.

cc) Differentiate between DDL and DML in SQL

DDL (`CREATE`, `ALTER`, `DROP`, `TRUNCATE`) defines and modifies schema objects; its effects are structural and often auto-commit. DML (`SELECT`, `INSERT`, `UPDATE`, `DELETE`) manipulates the data within those structures and is typically transactional, supporting `COMMIT/ROLLBACK` semantics.

dd) What do you mean by CSS?

CSS (Cascading Style Sheets) is a stylesheet language that describes the presentation of HTML/XML documents. It separates content from design and supports selectors,

cascade/inheritance, layout models (Flexbox, Grid), responsive media queries, and variables (custom properties).

ee) Write the types of Namespaces in XML.

Namespaces can be default (`xmlns='URI'` applied to unprefix elements within scope) or prefixed (`xmlns:ns='URI'` used as `ns:tag`). They are typically declared at the root element to qualify element and attribute names and prevent collisions across vocabularies (e.g., SVG + XHTML in one document).

ff) Write a note on DHTML.

DHTML marries HTML structure, CSS presentation, and JavaScript behavior to produce interactive pages. Using DOM scripting and CSS manipulation, DHTML can implement effects like dropdowns, in-page validation, and animations without reloading the page; modern SPAs extend these ideas with frameworks and virtual DOMs.